

第 8 集 · 约 17 分钟

构建与发布： 又小又快的 exe

同一份代码，换个编译方式就差 150 倍

配套阅读：书的第 8 章



这一集你会学到

1. debug 和 release 差多少：同一份加密代码，8 MiB/s 对约 1200 MiB/s
2. release 的四个旋钮：LTO、codegen-units、strip、opt-level，各省多少
3. 泛型在调用方编译：为什么只给依赖开优化不够
4. `include_bytes!` 和 `macro_rules!`：把网页编进 exe
5. 一条命令出成品，以及卸载脚本差点误删的东西

先看一句吐槽

“我测了一下，Rust 写的加密只有 8 MiB/s，还不如 Python。”

他测的是 debug 构建。同一份代码加上 `--release`，约 1200 MiB/s，差 150 倍。

release 配置的四个旋钮

Cargo.toml (工作区根目录)

```
1 [profile.release]
2 opt-level = 3      # 优化级别: 0 不优化, 3 最快; "s"/"z" 为体积优化
3 lto = true        # 链接时优化: 把所有 crate 合在一起再优化
4 codegen-units = 1 # 整个 crate 作为一个单元编译: 慢一点, 但优化得更彻底
5 strip = true      # 去掉符号信息
```

逐个打开，exe 怎么变



本机 i9-12900H，每一行都是真实构建出来的（书第 8 章）

动手：拧旋钮，看体积和速度

- debug (cargo build)
- A: cargo 默认 release
- B: + lto = true
- C: + codegen-units = 1
- D: + strip = true (项目在用)
- E: D + opt-level = "z"

```
[profile.release]

1  opt-level = 3
2  lto = true
3  codegen-units = 1
4  strip = true
```



strip 在 Windows 上只省了 512 字节：符号本来就在 .pdb 里

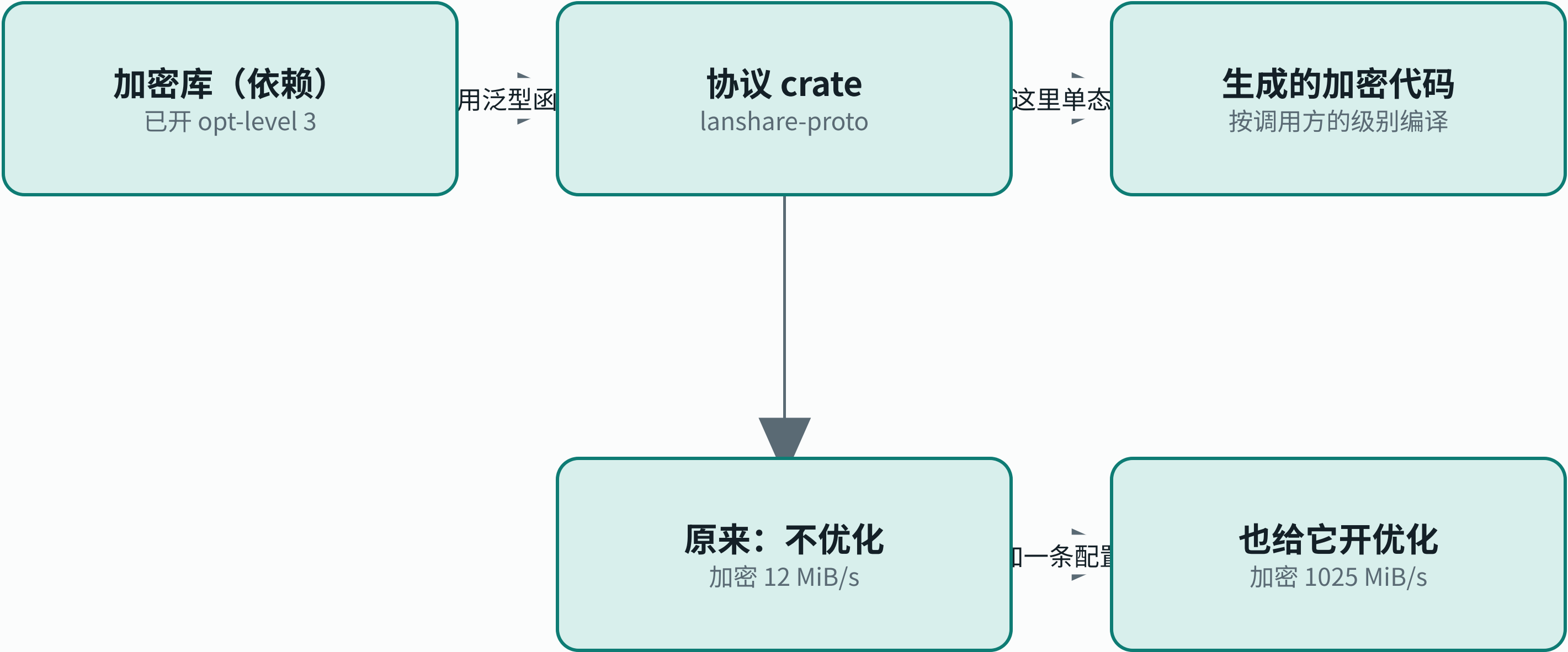
数字来自书第 8 章在开发机上的实测 (i9-12900H)。没单独测速的配置写“没单独测”。后来安全加固加了代码，现在的 exe 约 2.3 MiB。

开发构建也想快一点

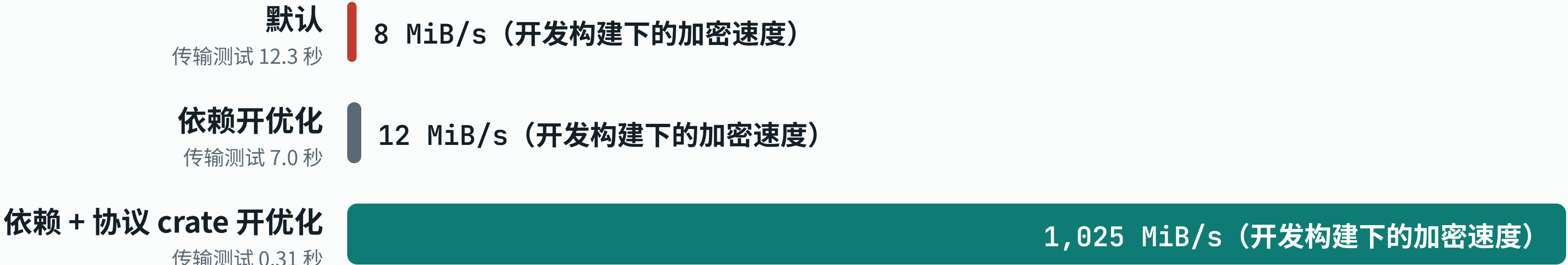
Cargo.toml

```
1 [profile.dev.package."*"]      # "*" = 所有依赖 (不含工作区自己的 crate)
2 opt-level = 3
3
4 [profile.dev.package.lanshare-proto]
5 opt-level = 3
```

泛型在调用方编译



一次真实的提速



本机 i9-12900H 实测（书第 8 章）

网页文件：运行时读 vs 编译时嵌

Python v1 · server.py

```
1 WEB_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "static")
2 # ...
3 with open(os.path.join(WEB_DIR, "index.html"), "rb") as f:
4     page = f.read()
5 self._send_bytes(200, page, "text/html; charset=utf-8")
```

Rust v2 · server/assets.rs

```
1 macro_rules! asset {
2     ($name:ident, $file:literal, $mime:literal) => {
3         pub(crate) async fn $name() -> Response {
4             let body: &'static [u8] = include_bytes!($file);
5             let mut response = body.into_response();
6             response.headers_mut().insert(header::CONTENT_TYPE, $mime.parse().unwrap());
7             response
8         }
9     };
10 }
11
12 asset!(index, "index.html", "text/html; charset=utf-8");
13 asset!(app_js, "app.js", "text/javascript; charset=utf-8");
14 asset!(proto_js, "proto.js", "text/javascript; charset=utf-8");
15 asset!(style_css, "style.css", "text/css; charset=utf-8");
16 asset!(wasm, "lanshare.wasm", "application/wasm");
17 asset!(favicon, "icon.ico", "image/x-icon");
```

一条命令出成品

Windows PowerShell

```
PS D:\lanshare> powershell -NoProfile -ExecutionPolicy Bypass -File build.ps1
==> build wasm
==> clippy
==> cargo test
==> node tests (debug exe)
==> release build
==> smoke test (release exe)
Done: dist\LanShare.exe (2282 KB, wasm 93.5 KB)
```

卸载脚本差点误删

```
1 # v1: 无条件删除任务栏固定
2 $Pinned = Join-Path $env:APPDATA "...\\User Pinned\\TaskBar\\局域网快传 LanShare.lnk"
3 Remove-Item $Shortcut, $Pinned -Force -ErrorAction SilentlyContinue
4
5 # v2: 只删指向本次卸载的 exe 的快捷方式
6 function Remove-OwnShortcut($path) {
7     if (-not (Test-Path $path)) { return }
8     $target = (New-Object -ComObject WScript.Shell).CreateShortcut($path).TargetPath
9     if ($target -eq $Exe) { Remove-Item $path -Force } else { Log "保留 $path (它指向 $target) " }
10 }
```

真实踩坑

编译报 failed to remove file ...\\LanShare.exe

测试用的 LanShare 还在运行，Windows 不许覆盖正在运行的 exe。我紧接着重启服务，跑的其实是旧 exe

exe 里装的会不会是旧的 WASM?

互通测试从运行中的 exe 下载每个资源，和仓库文件比哈希。故意改 style.css、不重编，结果 pass 8, fail 1

安装脚本怎么测才不弄坏自己的电脑?

把 LOCALAPPDATA、APPDATA 指到临时目录。沙箱里走完安装、运行、更新、卸载，17 项检查全过

build.ps1 只用 ASCII

PowerShell 5.1 读没有 BOM 的脚本按本地代码页解析，中文会乱；要显示中文的 install.ps1 就存成 UTF-8 带 BOM

选择题

`opt-level = "z"` 能让 exe 再小 27%，项目为什么不用？

- A "z" 编出来的程序容易出 bug
- B 加密慢了 4 倍多，对一个以快为卖点的工具不划算
- C 用了 "z" 就跑不满千兆网
- D 编译时间变长了

选择题

`opt-level = "z"` 能让 exe 再小 27%，项目为什么不用？

A "z" 编出来的程序容易出 bug

不会。它只是换了优化目标，程序行为不变，问题在速度。

B 加密慢了 4 倍多，对一个以快为卖点的工具不划算

对。实测加密从约 1200 MiB/s 掉到约 270 MiB/s，只为省 0.6 MB。体积优化更适合对大小很敏感、对速度不敏感的场景。

C 用了 "z" 就跑不满千兆网

跑得满。千兆网线的上限约 119 MiB/s，两种配置都超过它。真正的考虑是加密占的 CPU：电脑还要同时收发、写盘。

D 编译时间变长了

恰恰相反，实测 E 编译 26 秒，比现在用的 D 的 36 秒还快。

选择题

只写了 `[profile.dev.package."*"]` 给依赖开优化，debug 下加密为什么还是很慢？

- A `"*"` 不包含加密库
- B 泛型在调用方 crate 里实例化，按调用方的优化级别编译
- C 开发构建带着整数溢出检查，拖慢了速度
- D 开发构建的优化设置不生效，必须加 `--release`

选择题

只写了 `[profile.dev.package."*"]` 给依赖开优化，debug 下加密为什么还是很慢？

A `"*"` 不包含加密库

包含。`"*"` 指所有依赖，加密库也在里面。可它的泛型函数不在它自己那里编译，所以它的设置管不着。

B 泛型在调用方 crate 里实例化，按调用方的优化级别编译

对。调用加密库的是 `lanshare-proto`，它没开优化，生成的加密代码也没优化。给它也开上，就到了 1025 MiB/s。

C 开发构建带着整数溢出检查，拖慢了速度

给 `lanshare-proto` 开优化以后，溢出检查照样开着，速度却到了 1025 MiB/s。慢的原因不是它。

D 开发构建的优化设置不生效，必须加 `--release`

`[profile.dev.package...]` 就是专门给开发构建配的，加完实测确实变快了。

想一想

`build.ps1` 测完 debug 版，为什么还要拿 release 版 exe 把互通测试再跑一遍？测过 debug 不够吗？

想一想

`build.ps1` 测完 debug 版，为什么还要拿 release 版 exe 把互通测试再跑一遍？测过 debug 不够吗？

不够。release 和 debug 是用不同编译选项生成的两个程序，优化级别、LTO 都不一样；而且 release 版里内嵌的，是这次刚编好、刚拷进去的 WASM。只测 debug，等于没测真正发给用户的那个文件。

这一集我们学了

- 性能只看 release 构建：同一份加密代码，debug 8 MiB/s，release 约 1200 MiB/s
- LTO、codegen-units 各省一点，strip 在 Windows 上几乎没用；opt-level = "z" 小 27% 但慢 4 倍多
- 泛型在调用方 crate 里按调用方的优化级别编译，所以协议 crate 也要开优化
- `include_bytes!` 把网页编进 exe：文件不在就编译失败，不用等到运行时
- 一条命令出成品，每步看退出码，最后拿正式版 exe 真跑一遍

下一集

下一集：请外人来攻击它。修好了，复验时却发现修复本身带来了新问题